<Name-of-Software-Application>

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 7

Document Revision History

Version	Date	Author	Comments
1.0	03-26-2026	Ethan, Swift	Initial creation of software design document, including UML diagram and implementation of Game, Team, and Player classes.
2.0	4-10-2026	Ethan, Swift	Added evaluation of operating platforms and updated recommendations for part two of the Project.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The purpose of this software design is to create a game management application that allowed users to organize and manage games, teams, and players. The main problem I solved is how to efficiently store and access game data while ensuring that each object is unique with it's own ID.

To address this, the design uses object-oriented programming principles along with a singleton pattern to ensure that only one instance of the "GameService" class exists. This allowed for consistent access to game data across the application. The system is designed to be simple, scalable, and easy to maintain, providing a clear structure for managing relationships between the games, the teams, and the players.

Requirements

The client required a system that could create and manage game data, including the teams and players, while ensuring that each object has a unique identifier (ID). The system must also prevent duplicate entries.

From a technical perspective, the application must use object-oriented programming concepts such as classes, inheritance, and encapsulation. It should also implement the singleton design pattern to ensure that only one instance of the service class is used throughout the program. Additionally, the system should be easy to expand in the future if more features are added.

Design Constraints

A major design constraint is that the application must operate in a web-based distributed environment, This requires careful tailoring of shared resources, which is why the singleton pattern is so important.

Another constraint is maintaining unique identifiers for each game, team, and player. This impacts how objects are created and stored within the complex system. Additionally, the design must remain simple and readable, as it is intended for development and expansion.

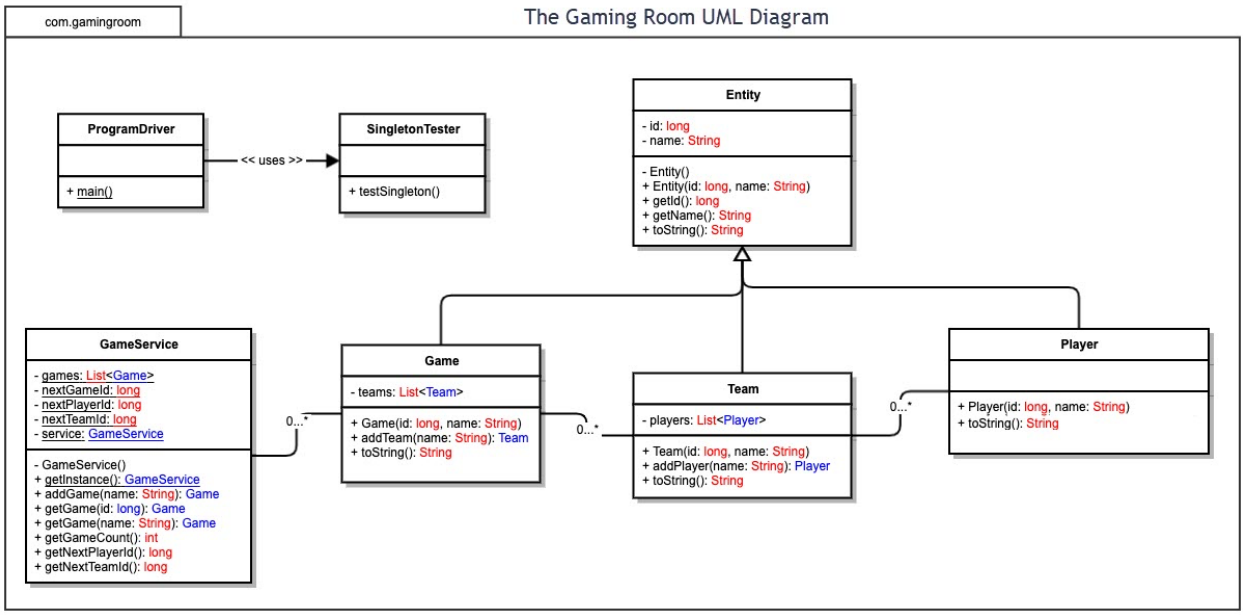
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The domain model shows the main classes used in the game application and how they interact with one another. The GameService class controls the overall game data and acts as the central hub for creating and retrieving particular games. A Game can contain multiple Team objects, and each Team can contain multiple Player objects. This shows a clear relationship between the classes and helps organize the data in a logical way.

The diagram also demonstrates object-oriented programming principles such as inheritance, encapsulation, and abstraction. The Entity class serves as a base class that keeps the shared ID and name fields, while Game, Team, and Player inherit from it instead of repeating the same code..



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	I determined that Mac can be used for server-side hosting, but it is not commonly used for larger web applications. It is more expensive due to the hardware used and licensing, which makes it less practical for a making/extending an application. While it's stable and secure, it does not scale as easily as other platforms.	Linux is definitely the best option for server-side hosting because it's open-source and doesn't require any licensing fees (making it very cost-effective). It is widely used for web-based applications and supports big, scalable environments that can handle thousands of players. On top of that, Linux also works well with cloud platforms and other common web technologies, making it ideal for a distributed system like this game.	Windows can support server-side hosting and is user-friendly, especially for people familiar with Microsoft tools in general, this will be an easier transition. However, it typically requires licensing fees, which increases the cost. Though it integrates well with other Microsoft services, it's generally less flexible used for large-scale web hosting compared to Linux.	Mobile devices are not suitable for server-side hosting at all because they do not have the resources or stability needed to support such a large number of users. Instead, they are designed to access applications rather than actually host them. These are for the client-side.
Client Side	Mac provides a reliable user experience and could support modern web browsers, making it suitable for accessing a web-based applications. Though, it's more spendy than other platforms, which could limit accessibility for some people.	Linux can be used on the client side and supports web browsers, but it may have compatibility issues with some specific applications. It's also less user-friendly for a general user, which could impact accessibility for a wider audience.	Windows is widely used and supports most web browsers/ applications, which makes it a strong choice for client-side accessibility. It is user-friendly and accessible to many users, which helps ensure the application can reach a wide audience.	Mobile devices are also very convenient for accessing web-based applications through browsers. They allow users to play the game from anywhere which is a great perk, though they may have some performance limitations compared to desktop systems.

Development Tools	Mac supports development tools such as Eclipse, Visual Studio Code, etc.; and is commonly used by developers. But the cost of Apple hardware can be higher, which may bump up the development expenses.	Linux supports a wide scope of development tools and is powerful for programming, especially for the server-side development. It is also free to use, which helps reduce costs. Though, it may require more technical expertise from the development team.	Windows supports popular development tools such as Visual Studio and is beginner-friendly, making it easier for development teams to work with. Though, licensing costs for certain tools and the operating system itself can bump up the overall expenses.	Mobile devices are mainly used for testing applications rather than full development. Developers may need separate tools and teams to ensure the application works properly across different mobile platforms, which can increase development time and complexity.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: The best operating platform for The Gaming Room is Linux, in my opinion, because it is cost-effective, scalable, and highly reliable for web-based distributed applications. Since Linux is open-source, it eliminates licensing costs and allows greater flexibility for customization. Additionally, Linux is widely sed in cloud environments such as AWS and Azure, making it easier to deploy and scale the application as the number of users grows. Its strong community support and compatibility with server technologies make it the most practical choice for long-term expansion.
2. **Operating Systems Architectures**: Linux uses a modular and layered architecture, which separates the kernel from user-level processes. This allows efficient multitasking and better resource management when handling multiple users at once. The kernel manages system resources such as CPU, memory, and device access, while user processes run independently, improving system stability. This architecture is ideal for a distributed application because it supports concurrency, process isolation, and high availability across multiple systems.
3. **Storage Management**: A relational database management system such as MySQL should be used for storage management. MySQL allows structured storage of game, team, and player data while ensuring data integrity through constraints and relationships. It also supports indexing and query optimization, which improves performance when retrieving large amounts of data. Additionally, using centralized database storage allows consistent access across distributed systems and supports scalability as the application grows.
4. **Memory Management**: Linux uses advanced memory management techniques such as virtual memory, paging, and memory protection to efficiently allocate system resources. Virtual memory allows the system to use disk space as an extension of RAM, ensuring that applications continue to run even under heavy load. Paging helps divide memory into manageable sections, while process isolation prevents one application from interfering with another. These techniques help maintain stability, performance, and responsiveness for the game application.
5. **Distributed Systems and Networks**: The application should use a client-server architecture within a distributed system, where clients (users on desktop or mobile devices) communicate with centralized servers over the internet. This allows multiple users to access the game simultaneously from different locations. To ensure reliability, the system can use load balancing and redundancy to handle high traffic and prevent downtime. Network communication should rely on standard protocols such as HTTP/HTTPS to ensure compatibility across platforms.
6. **Security**: Security should be implemented through multiple layers, including encryption, authentication, and access control. Data transmitted between clients and servers should use HTTPS to protect against interception. User authentication systems should ensure that only authorized users can access the application, while role-based access control can limit permissions within the system. Additionally, secure storage practices such as hashing passwords and protecting database access help prevent data breaches and maintain user trust.